

English → Indian Sign Language Avatar Demo

Goal	Develop a system for showing a 2D animated ISL avatar in the browser and on mobile devices.
Approach	LLM-prompted text → gloss + dictionary-lookup keypoint animation. No model training.
Vocabulary	ISLRTC dictionary — Indian Sign Language Research and Training Centre, 3rd ed. (2021), 10,000 terms across six categories.

The Pipeline

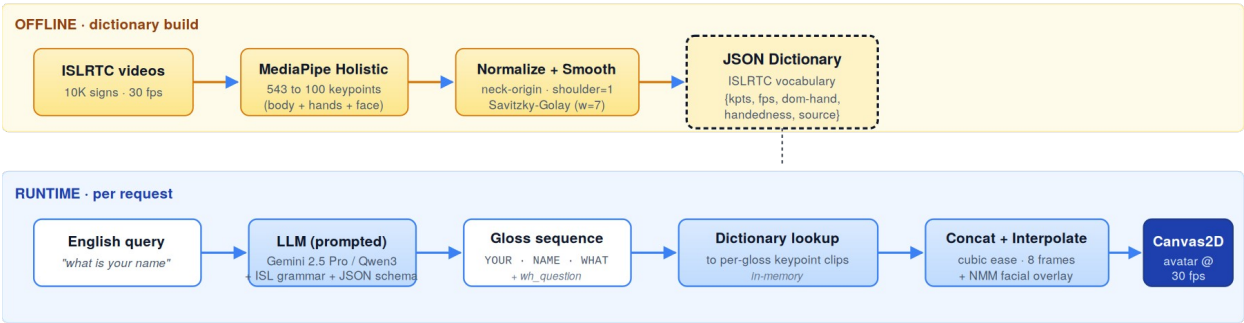


Figure 1. End-to-end pipeline. Top band runs once during dictionary build; bottom band runs per request.

Why This Architecture

The text → gloss → pose-lookup → avatar pattern is the canonical Sign Language Production design (Stoll 2020, Saunders 2020–22, Moryossef 2023). LLM prompting for the text-to-gloss step is now validated as competitive with trained baselines on low-resource sign languages (Guo 2025, Sincan 2024, AulSign 2025). It is the only approach that can be shipped end-to-end without paired video and text training data, which does not exist for ISL at useful scale. The gloss intermediary acts as a clean symbolic interface between the language model and the dictionary-driven animation engine.

System Architecture

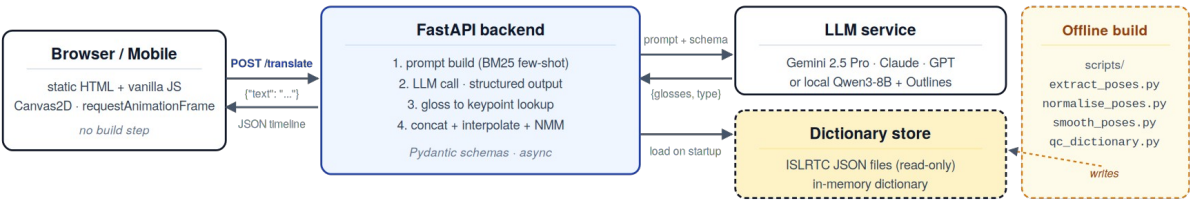


Figure 2. Component view. The browser is a thin client (just plays a timeline); the backend is stateless; the dictionary is read-only and built offline.

Task-wise Breakdown

The build is staged so that each task delivers a working artifact and de-risks the next one. Tasks 1–2 establish the rendering and normalization pipeline at minimum scale. Task 3 hardens the runtime against arbitrary hand-authored gloss sequences. Task 4 scales the dictionary. Task 5 finally adds the language-understanding component.

1

Render a single gloss “APPLE” as an animated avatar

Establishes the skinning pipeline end-to-end. Manually capture or extract one ISLRTC video for the sign APPLE; run MediaPipe Holistic to produce a keypoint time series; serialize to a JSON clip; load it in the browser and render the stick-figure avatar at 30 fps with requestAnimationFrame.

Goal: a single hardcoded clip plays correctly in the browser. No backend, no LLM, no dictionary — just keypoints → Canvas2D renderer.

2

Add “APPLE + RED” set up normalization and interpolation

Two glosses surface every concatenation issue at minimum scale. Implement neck-origin translation and shoulder-width scale normalization on each clip; concatenate APPLE then RED; synthesize 8 cubic-eased transition frames between them in keypoint space. Verify visually that the hand does not teleport between signs.

Goal: APPLE RED plays as a single fluent motion. The normalization and interpolation modules from this task are the same code used for the full vocabulary later — get them right here.

3

Extend to a 20-word vocabulary and verify by hand

Build a 20-gloss subset of the ISLRTC dictionary (e.g. I, YOU, MY, HELLO, THANK-YOU, NAME, WHAT, HOW, YES, NO, NOT, APPLE, RED, GOOD, WATER, FRIEND, MOTHER, WANT, EAT, KNOW). Hand-author the gloss sequences for the ten reference sentences below and verify each one renders correctly. This locks the runtime pipeline before any LLM is involved.

Reference sentences (ISL grammar: Time-S-O-V, drop copulas, wh-final, negation after verb):

ISL gloss sequence	English meaning
HELLO	Hello
THANK-YOU	Thank you
YOU NAME WHAT	What is your name?
I APPLE WANT	I want an apple
APPLE RED	The apple is red
I RED APPLE WANT	I want a red apple
I KNOW NOT	I don't know
YES I WATER WANT	Yes, I want water
MOTHER GOOD	Mother is good
FRIEND HOW	How is your friend?

Goal: all ten sentences play recognizably with smooth transitions. Failures here are pipeline bugs, not LLM bugs .

4

Extend to the entire ISLRTC dictionary

Run the dictionary-build pipeline (extract → normalise → smooth → JSON) over every ISLRTC video. Confirm that the runtime pipeline scales.

Goal: any gloss in the ISLRTC dictionary can be looked up by name and played correctly. Hand-authored gloss sequences over arbitrary vocabulary now work end-to-end.

5

Add LLM gloss support

Wire in the LLM-prompted text-to-gloss stage. System prompt encodes ISL grammar rules (Time-S-O-V, drop copulas/articles, wh-final, negation-after-verb). Few-shot examples retrieved by BM25 from a curated pool. Output constrained to the dictionary vocabulary via provider-side JSON schema (Gemini) or Outlines regex (local Qwen3). OOV strategy: semantic substitution → fingerspelling fallback → omission with warning.

Goal: user types an English sentence in the browser and the avatar signs it. Reference sentences from Task 3 should now be reachable from the English source side, not just the gloss side.

Expected Split

The project can be split along the rendering/data seam, with Person A owning the browser rendering pipeline, runtime synthesis, interpolation, Canvas2D, requestAnimationFrame, and NMM overlays, while Person B owns the offline dictionary pipeline, MediaPipe extraction, QC tooling, ISLRTC scale-up, and LLM structured-output work. The five tasks should be staged so each artifact becomes the foundation for the next, with Task 3 serving as the critical integration checkpoint where A's runtime consumes B's 20-gloss subset and executes the ten reference sentences end-to-end. A realistic two-person timeline is 6–8 weeks: Tasks 1–2 in weeks 1–2, Task 3 in weeks 3–4, Task 4 in weeks 5–6, and Task 5 in weeks 7–8, with the rule that no downstream work begins until the current task passes its acceptance check. Metrics should be instrumented from week 1, including p50/p95 latency breakdowns for every /translate stage, single-gloss recognizability scoring before sentence-level evaluation, and co-articulation quality checks focused on transition-frame velocity and wrist-trajectory inspection. If scope slips, vocabulary or LLM ambition should be reduced, but Tasks 1–3 must remain intact because they form the core load path for the entire system.
